American International University-Bangladesh (AIUB)

Faculty of Science and Technology

Department of Computer Science

Web Technologies

CSE 3215

PROJECT REPORT

Project Title:

PharmaQuick – Online Medicine Shop

Semester: Spring 25-26

Submission Date: May 19, 2026

Section: H

Submitted To:

Wahidul Alam Riyad

Lecturer, Faculty

Department of Computer Science

American International University-Bangladesh

Group Information

Group Number: 07

Project Title: PharmaQuick – Online Medicine Shop

Student Information:

Full Name: Md. Nabil Adibur Rahman

Student ID: 23-53187-3

Program: Bachelor of Science is Computer Science and Engineering

Email: 23-53187-3@student.aiub.edu

Signature: _____

Declaration: We hereby declare that this project report is our original work. Any external resources or references used have been properly cited. We have not engaged in any form of academic dishonesty.

Marks Sheet

To be filled by the instructor only. Students must NOT write in this section.

Group No.		**Section**	
Member 1 ID			

CO3: Illustrate Multi-Tier Web Application for Targeted Society					
Feature Implementation (0-3)			**Feature Implementation (0-3)**		**Completeness (0-3)**
Basic Web Security	**UI (HTML/CSS)**	**Feature Completeness**	**DB**	**Auth (Session/cookie)**	**MVC**
CO4: Coordinate with Group Members to Develop Multi-Tier Application					
Technical Knowledge (0-3)			**Collaborative Teamwork (0-3)**		**Communication/Promptness (0-3)**
JS validation	**PHP Validation**	**Ajax/JSON**	**Git Contribution**		**VIVA**
Expected Outcome (0-2)		**CO3 Total**	**CO4 Total**		**Project Total**

Evaluated By: _____ **Date:** _____

Course Teacher Signature: _____

Table of Contents

Chapter 1: Project Overview

1.1 Project Description

PharmaQuick is a multi-tier web application that simulates an e-commerce medicine purchasing platform. It solves the inconvenience of physically visiting pharmacies by allowing customers to browse, search, and order medicines online from multiple vendors, all from a single interface.

The system serves three distinct user roles:

- Customer - registers, logs in, browses medicines, manages a cart, places orders and tracks order history.
- Vendor (Pharmacy) - logs in to manage their own medicine listings (add, edit, delete) and view their order history.
- Admin - oversees the entire platform, manages all medicines, categories, customers, vendors, orders and views the purchase history each customer.

The application is developed as a group academic project for CSE 3215 (Web Technologies) at AIUB, demonstrating full-stack web development concepts including MVC architecture, relational databases, session-based authentication, AJAX, and collaborative Git workflows.

1.2 Technologies Used

Layer	Technology / Tool	Purpose
Frontend	HTML5, CSS3	UI structure and responsive layout
	JavaScript (Vanilla)	Client-side form validation, AJAX calls, dynamic cart updates
Backend	PHP 8.2	Server-side logic, MVC pattern routing, session management
Database	MySQL	Persistent data storage for users, medicines, orders, payments
Auth	PHP Sessions & Cookies	Login state management; Remember Me via cookie (30-day)
Version Control	Git / GitHub	Feature-branch workflow with Pull Requests
Server	XAMPP / Apache	Local development environment

1.3 System Architecture / MVC Diagram

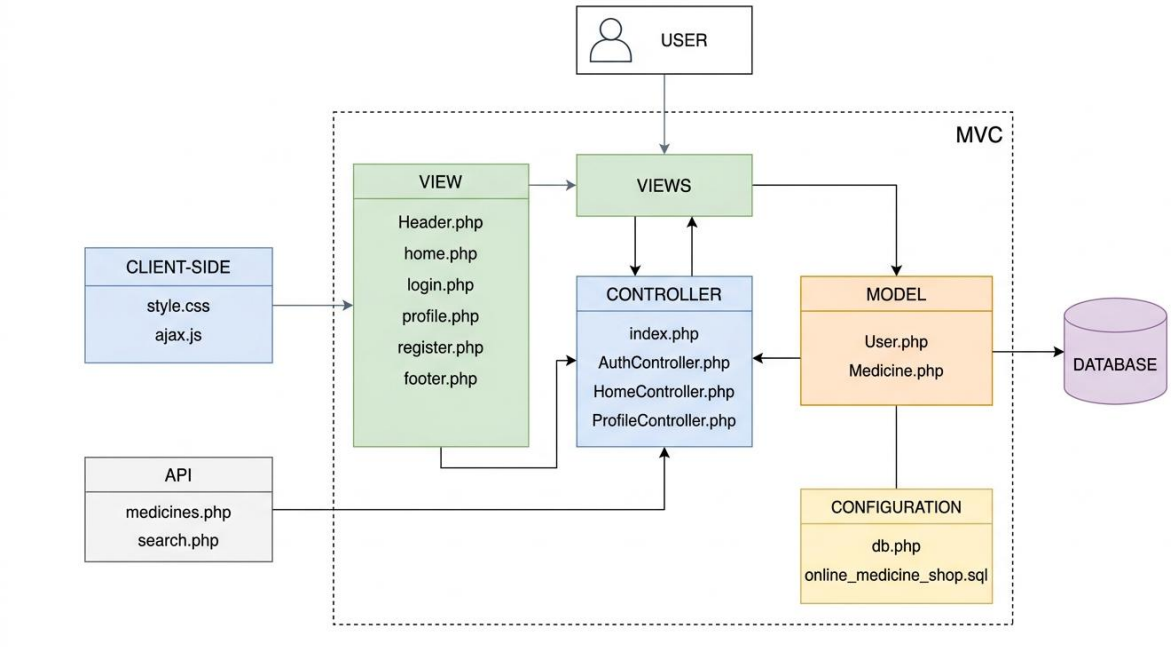


Fig: MVC Architecture Diagram

1.4 Database Schema

Table Name	Field Name	Data Type	Description
users	id	INT (PK)	Auto-increment user identifier
	name	VARCHAR(100)	User's full name
	email	VARCHAR(255)	Unique email for login
	password_hash	VARCHAR(255)	Bcrypt hashed password
	role	ENUM	'admin', 'customer', 'vendor'
	address	TEXT	User's shipping address
	phone	VARCHAR(20)	Contact number
	profile_picture	VARCHAR(255)	Path to profile image
	created_at	DATETIME	Account creation timestamp
medicines	id	INT (PK)	Auto-increment medicine ID
	name	VARCHAR(100)	Medicine name
	category_id	INT (FK)	References categories.id
	vendor_name	VARCHAR(100)	Vendor/store name
	price	DECIMAL(10,2)	Medicine price (BDT)
	availability	INT	Stock quantity
	description	TEXT	Medicine details
categories	id	INT (PK)	Category identifier
	name	VARCHAR(100)	Category name
	category_type	ENUM	'solid' or 'liquid'

Chapter 2: Git Contribution & Task Allocation

2.1 GitHub Repository Link

Repository URL: https://github.com/nabil-adib/online-medicine-shop.git

2.2 Git Contribution Screenshot

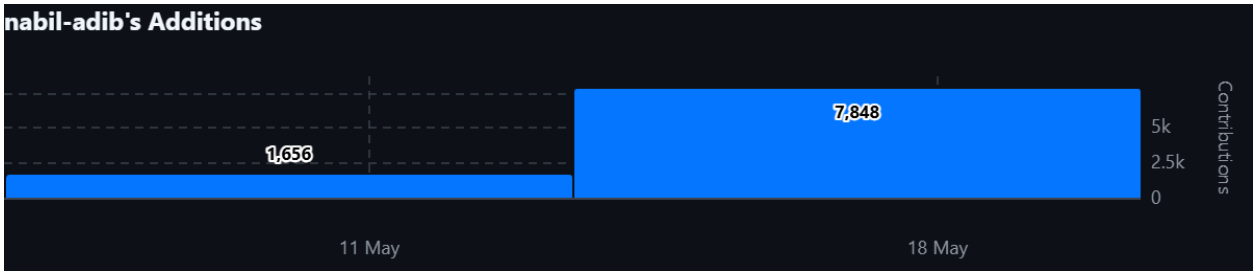


Fig: GitHub Contribution Graph

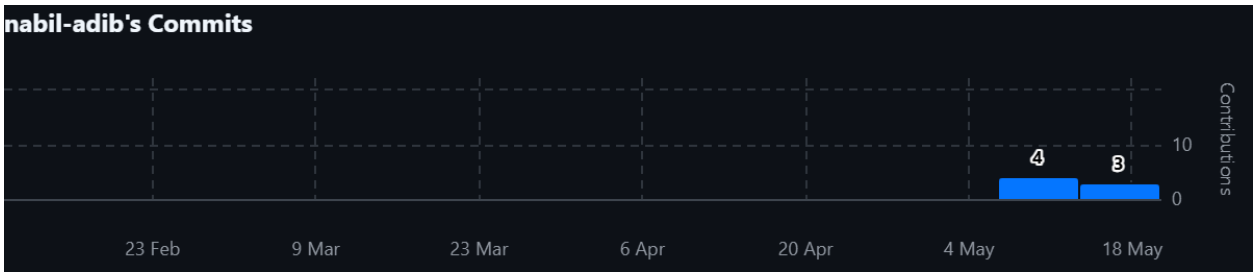


Fig: GitHub Commit History

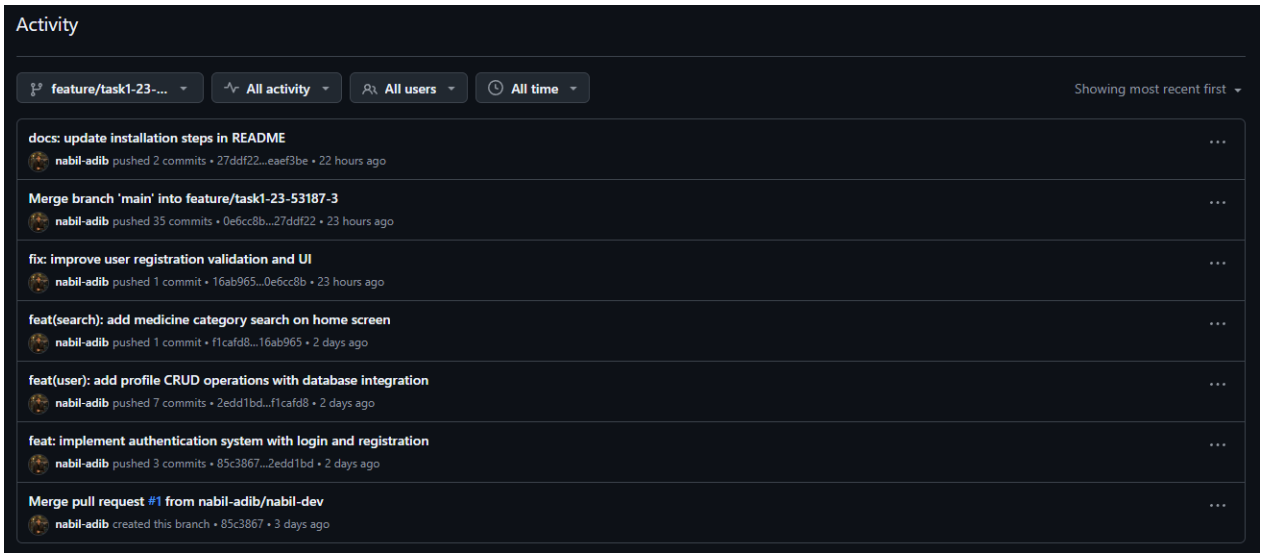


Fig: GitHub Commit Activity

Chapter 3: Features, Source Code & Screenshots

All features documented in this section were implemented as part of the assigned role of Customer-Side Module Developer, focusing on authentication, profile management and medicine browsing with AJAX-based search and filtering.

Feature 1: User Registration & Input Validation

Description

Allows new users to create an account by providing their name, email, password, address, and phone number. The feature enforces both client-side (JavaScript) and server-side (PHP) validation before creating the record in the database. Supported roles at registration are 'customer' and 'vendor'.

Relevant Source Code

File: Online_Medicine_Shop/controllers/AuthController.php

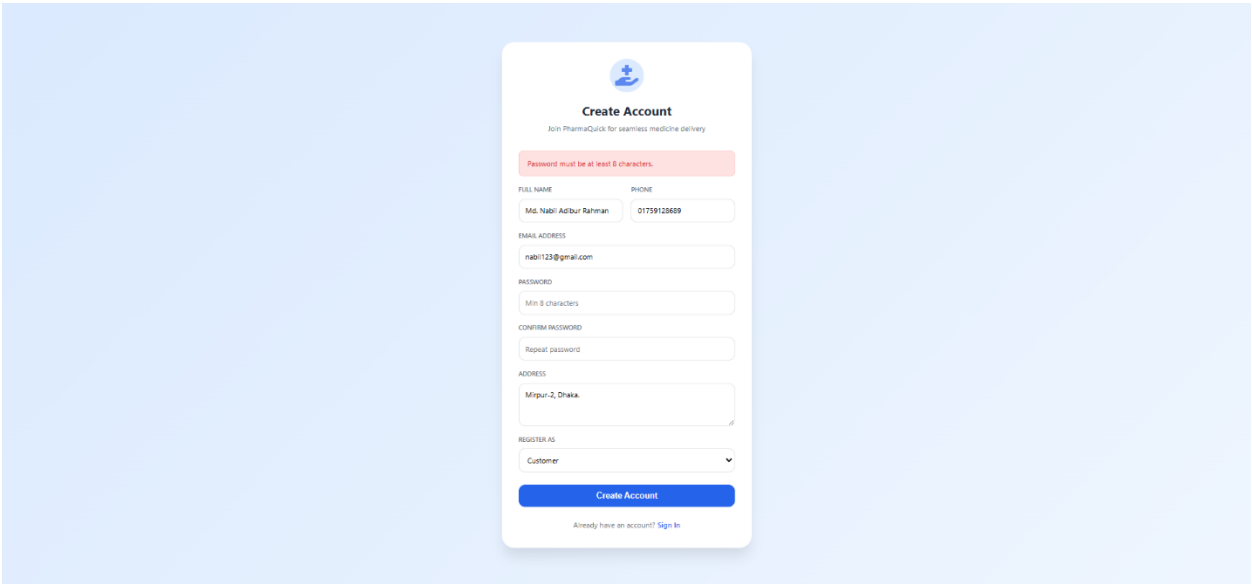
```
function register_ctrl($conn) {
    $error = '';
    $success = '';
    // Preserve form data for repopulation on error
    $old_name = ''; $old_email = ''; $old_address = ''; $old_phone = '';
    $old_role = 'customer';
    // Process form submission
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $user_name = trim($_POST['name'] ?? '');
        $user_email = trim($_POST['email'] ?? '');
        $user_password = $_POST['password'] ?? '';
        $confirm = $_POST['confirm_password'] ?? '';
        $user_address = trim($_POST['address'] ?? '');
        $user_phone = trim($_POST['phone'] ?? '');
        $user_role = trim($_POST['role'] ?? 'customer');
        // Store for form repopulation
        $old_name = $user_name;
        $old_email = $user_email;
        $old_address = $user_address;
        $old_phone = $user_phone;
        $old_role = $user_role;
        // Validation checks
        if ($user_name === '' || $user_email === '' || $user_password === '' ||
            $user_address === '' || $user_phone === '') {
            $error = 'All fields are required.';
        } elseif (!validate_name($user_name)) {
              $error = 'Name cannot contain numbers. Please use only letters and
spaces.';
        } elseif (!filter_var($user_email, FILTER_VALIDATE_EMAIL)) {
            $error = 'Please enter a valid email address.';
        } elseif (strlen($user_password) < 8) {
            $error = 'Password must be at least 8 characters.';
        } elseif ($user_password !== $confirm) {
            $error = 'Passwords do not match.';
        } elseif (!preg_match('/^\d{11}$/', $user_phone)) {
            $error = 'Phone number must be exactly 11 digits.';
        } elseif (email_exists($conn, $user_email)) {
            $error = 'This email is already registered.';
        } else {
            // Hash password and create user
            $password_hash = password_hash($user_password, PASSWORD_DEFAULT);
            $ok = create_user(
                $conn,
                htmlspecialchars($user_name),  // Prevent XSS attacks
                $user_email,
```

```
                $password_hash,
                htmlspecialchars($user_address),
                $user_phone,
                $user_role
            );

            if ($ok) {
                 $_SESSION['registration_success'] = "Account created successfully!
Please login with your credentials.";
                header('Location: index.php?page=login');
                exit;
            } else {
                $error = 'Registration failed. Please try again.';
            }
        }
}

    require 'views/register.php';
}
```

Code Explanation

- validate_name() uses preg_match('/[0-9]/', $name) to reject names containing digits.
- filter_var($email, FILTER_VALIDATE_EMAIL) is PHP's built-in email format checker.
- email_exists() runs a prepared SELECT statement in the users table to prevent duplicate registrations.
- password_hash($pass, PASSWORD_DEFAULT) stores the password as a bcrypt hash for security.
- htmlspecialchars() escapes name and address to prevent XSS before storing in the database.
- On success, a session flag is set and the user is redirected to the login page.

Screenshot(s)



Feature 2: User Login with Session & Remember Me Cookie

Description

Authenticates registered users using email and bcrypt password verification. On successful login, role-based session variables are set and the user is redirected to the appropriate dashboard (admin, vendor, or customer home). A 'Remember Me' checkbox persists the email in a 30-day cookie.

Relevant Source Code

File: Online_Medicine_Shop/controllers/AuthController.php

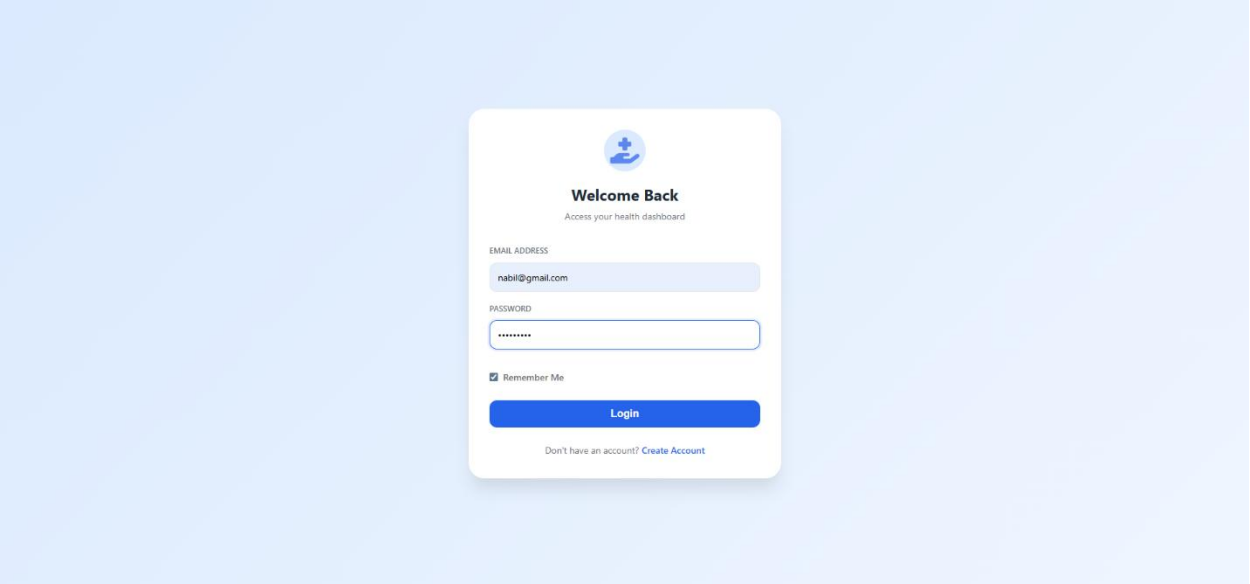
```
function login_ctrl($conn) {
    $error = '';

    // Pre-fill email if "Remember Me" cookie exists
    $prefill_email = $_COOKIE['remember_user'] ?? '';

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $user_email = trim($_POST['email'] ?? '');
        $user_password = $_POST['password'] ?? '';
        $remember = isset($_POST['remember']);  // Check if remember me is checked

        if ($user_email === '' || $user_password === '') {
            $error = 'Please fill in both fields.';
        } elseif (!filter_var($user_email, FILTER_VALIDATE_EMAIL)) {
            $error = 'Please enter a valid email address.';
        } else {
            $user = get_user_by_email($conn, $user_email);
            // Verify password against stored hash
            if ($user && password_verify($user_password, $user['password_hash'])) {
                // Set session variables for logged-in user
                $_SESSION['logged_in'] = true;
                $_SESSION['user_id'] = $user['id'];
                $_SESSION['user_name'] = $user['name'];
                $_SESSION['user_email'] = $user['email'];
                $_SESSION['user_role'] = $user['role'];
                $_SESSION['user_phone'] = $user['phone'];
                $_SESSION['user_address'] = $user['address'];
                $_SESSION['profile_picture'] = $user['profile_picture'];

                // Set persistent cookie for 30 days if "Remember Me" is enabled
                if ($remember) {
                      setcookie('remember_user', $user_email, time() + 86400 * 30,
'/');
                } else {
                     setcookie('remember_user', '', time() - 3600, '/');  // Delete
cookie
                }

                // Redirect based on user role
                switch ($user['role']) {
                    case 'admin':
                        header('Location: index.php?page=admin&action=dashboard');
                        break;
                    case 'vendor':
                        header('Location: index.php?page=vendor_home');
                        break;
                    case 'customer':
                    default:
                        header('Location: index.php?page=home');
                        break;
                }
                exit;
            } else {
                $error = 'Invalid email or password.';
```

```
            }
        }
    }

    // Pass prefilled email to login view for form autofill
    require 'views/login.php';
}
```

Code Explanation

- get_user_by_email() performs a parameterized SELECT query, preventing SQL injection.
- password_verify() checks the submitted password against the stored bcrypt hash
- On success, all user data (id, name, role, email, phone, address, profile_picture) is stored in $_SESSION.
- setcookie('remember_user', $email, time()+86400*30, '/') saves the email for 30 days; the login form pre-fills from $_COOKIE['remember_user'].
- Role-based redirect ensures admins land on the admin dashboard, vendors on vendor_home, and customers on the home page.

Screenshot(s)



Feature 3: Profile Management (Update Info, Change Password, Upload Avatar)

Description

Authenticated users can update their personal details (name, email, address, phone), change their password, and upload a profile picture (JPG/PNG/GIF, max 2 MB). All three actions have separate POST handlers under a single action field, with full server-side and client-side validation.

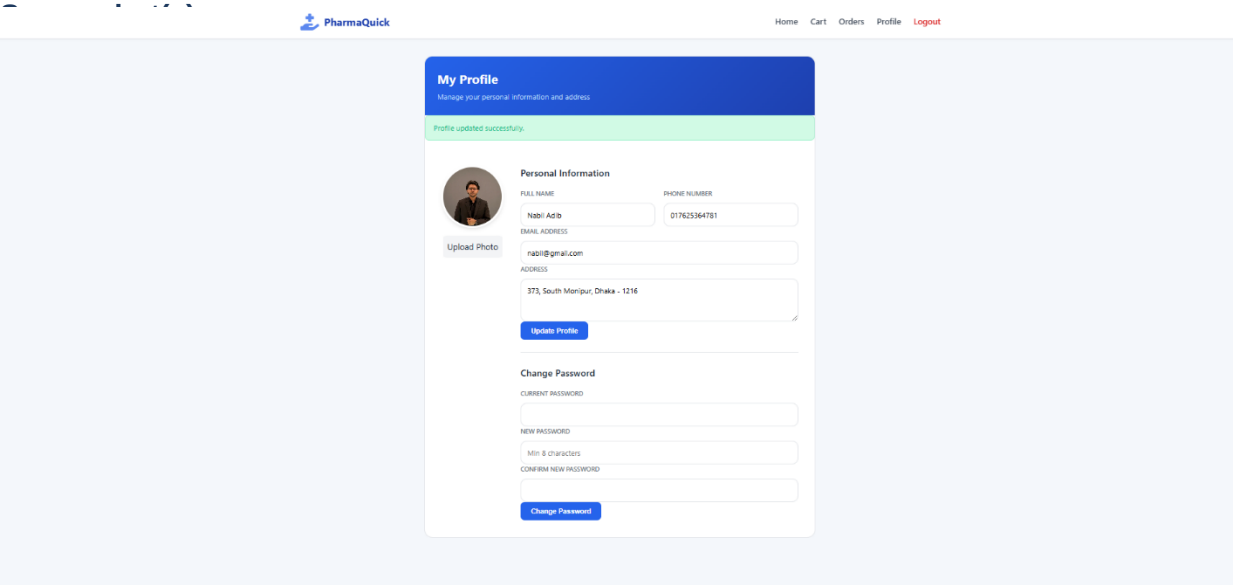
Relevant Source Code

File: Online_Medicine_Shop/controllers/ProfileController.php

```
function profile_ctrl($conn) {
    $user = get_user_by_id($conn, $_SESSION['user_id']);
    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $action = $_POST['action'] ?? 'update_info';
        if ($action === 'update_info') {
            // validate and update name, email, address, phone
            $result = update_user_info($conn, $_SESSION['user_id'],
                        htmlspecialchars($name), $email, ...);
            if ($result === 'email_exists') {
                $error = 'Email already used by another account.';
            }
        }
        if ($action === 'change_password') {
            if (!password_verify($current_pass, $user['password_hash'])) {
                $error = 'Current password is incorrect.';
            } elseif (strlen($new_pass) < 8) { ... }
            else {
                $hash = password_hash($new_pass, PASSWORD_DEFAULT);
                update_user_password($conn, $_SESSION['user_id'], $hash);
            }}
        if ($action === 'upload_picture') {
            $finfo = finfo_open(FILEINFO_MIME_TYPE);
            $mime  = finfo_file($finfo, $file_tmp);
            if (!in_array($mime, ['image/jpeg','image/png','image/gif'])) {
                $error = 'Only JPG, PNG, GIF allowed.';
            } elseif ($size > 2*1024*1024) {
                $error = 'Image must be under 2MB.';
            } else {
                $filename = 'profile_'.$user_id.'_'.time().'.'.$ext;
                move_uploaded_file($tmp, 'public/uploads/'.$filename);
                update_profile_picture($conn, $user_id, $path);
            }}}
    require 'views/profile.php';
}
```

Code Explanation

- update_user_info() checks whether the new email is already taken by a different user before executing the UPDATE query, returning 'email_exists' as a string signal.
- password_verify() validates the current password before any change is allowed, preventing unauthorized password resets.
- finfo_open(FILEINFO_MIME_TYPE) checks the actual file MIME type to prevent malicious uploads disguised as images.
- Uploaded files are renamed to profile_{user_id}_{timestamp}.{ext} to avoid filename collisions and directory traversal.
- Session variables are refreshed after a successful update so the fields reflects the new information immediately.


PharmaQuick
Home
Cart
Orders
Profile
Logout
My Profile
Manage your personal information and address
Profile updated successfully.
Upload Photo
Personal Information
FULL NAME
Nabil Adib
PHONE NUMBER
01762536478I
EMAIL ADDRESS
nabil@gmail.com
ADDRESS
373, South Moripur, Dhaka - 1216
Update Profile
Change Password
CURRENT PASSWORD
NEW PASSWORD
Min 8 characters
CONFIRM NEW PASSWORD
Change Password

Feature 4: Medicine Browsing with Dynamic Search & Filters

Description

The homepage shows medicines in a grid and allows real-time filtering by search text, vendor, category, and form factor (solid/liquid). All updates happen using AJAX without page reload for a smooth user experience.

Relevant Source Code

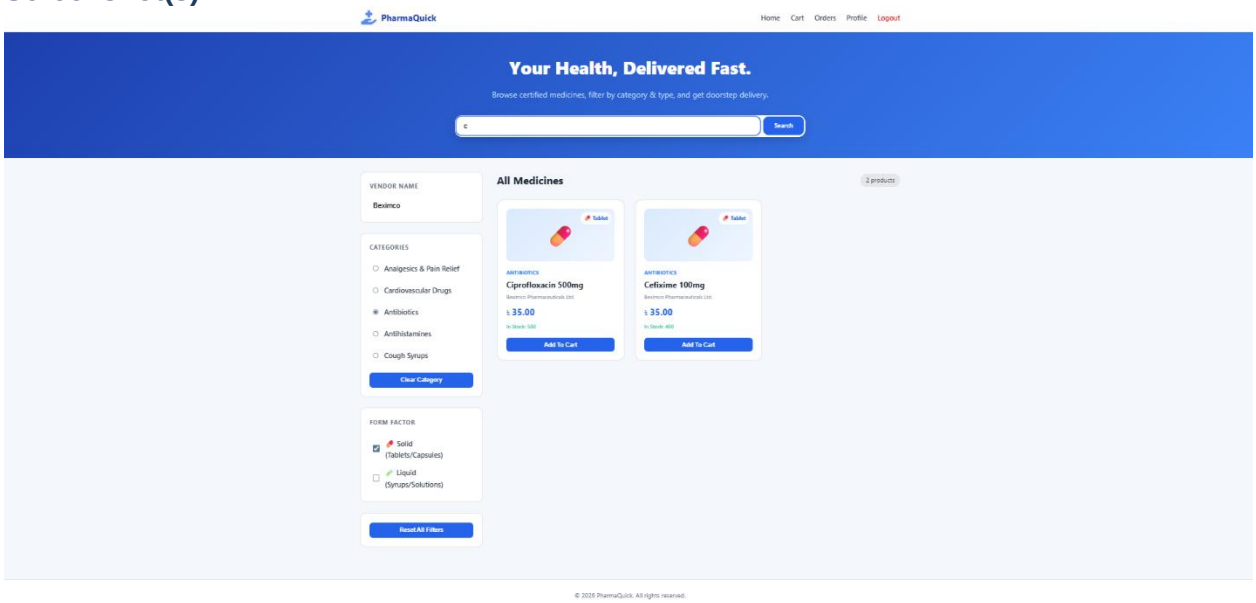
File: Online_Medicine_Shop/views/home.php

```
function searchMedicines() {
    // Get filter values from DOM elements
    let search = document.getElementById('searchInput').value;
    let vendor = document.getElementById('vendorFilter').value;
    let genre = activeCategory;
    let type = '';
    // Determine form factor filter selection
    let solidChecked = document.getElementById('filterSolid').checked;
    let liquidChecked = document.getElementById('filterLiquid').checked;

    if (solidChecked && liquidChecked) {
        type = 'all';
    } else if (solidChecked) {
        type = 'solid';
    } else if (liquidChecked) {
        type = 'liquid';
    }
    // Build API URL with query parameters
    let url = 'public/api/search.php?';
    if (search) url += 'q=' + encodeURIComponent(search) + '&';
    if (vendor) url += 'vendor=' + encodeURIComponent(vendor) + '&';
    if (genre) url += 'genre=' + encodeURIComponent(genre) + '&';
    if (type) url += 'type=' + encodeURIComponent(type);

    // Remove trailing ampersand if exists
    if (url.endsWith('&')) {
        url = url.slice(0, -1);
    }
    // Show loading state while fetching
         document.getElementById('productGrid').innerHTML    =    '<div    class="no-
results">Loading medicines...</div>';

    // AJAX GET request to fetch medicines
    fetch(url)
        .then(response => response.json())
        .then(data => {
            if (data.success) {
                allMedicines = data.medicines;

                // Extract and render categories only once (if empty)
                if (categoriesList.length === 0) {
                    extractCategories();
                    renderCategories();
                }
                renderMedicines();  // Display medicines on page
            } else {
                 document.getElementById('productGrid').innerHTML = '<div class="no-
results">Failed to load medicines</div>';
            }
        })
        .catch(error => {
            console.log('Error:', error);
               document.getElementById('productGrid').innerHTML  =  '<div  class="no-
results">Error connecting to server</div>';
        });}
```

Code Explanation

- Collects filter values from the UI (search box, vendor, and medicine type).
- Builds a dynamic URL depending on which filters are selected.
- Sends an AJAX GET request using fetch() to the backend API.
- Receives filtered medicine data in JSON format.
- Updates the page dynamically by calling renderMedicines() without reloading.
- The backend receives these parameters and runs a database query to return only matching medicines.

Screenshot(s)



Chapter 4: Conclusion & Future Work

4.1 Summary

For this project, I successfully implemented the Authentication and Seach & Filtering features of the PharmaQuick Online Medicine Shop System. The implemented features include user registration with input validation, secure login with session handling and “Remember Me” functionality, profile management with update, password change, and image upload capabilities, and a dynamic medicine browsing interface with AJAX-based search and filtering. Security was reinforced using password hashing, session management, prepared database queries, and file upload validation.

The primary goals achieved were

- ensuring secure authentication mechanisms
- maintaining data integrity through strict validation on both client and server sides
- improving user experience through responsive and asynchronous UI interactions

4.2 Challenges Faced

A key challenge was handling email uniqueness during profile updates while still allowing users to keep their existing email. This was solved by returning a custom email_exists() function was used to properly detect duplicate cases.

Another issue was secure file upload validation, where malicious files could bypass checks using fake extensions. This was fixed by validating the real file type using finfo_open(FILEINFO_MIME_TYPE) instead of relying on extensions.

Managing AJAX state updates in medicine filtering was also challenging, as UI changes needed to reflect instantly without losing filter selections. This was resolved using a centralized allMedicines array and a consistent renderMedicines() function.

4.3 Future Improvements

Email Verification & Password Reset: Email confirmation during registration and a secure password reset system via email links (e.g., PHPMailer with SMTP) would be implemented to enhance account security and recovery.

Order Tracking with Real-Time Notifications: Real-time order status updates would be introduced using push notification services (Firebase/WebPush), along with a visual order timeline and estimated delivery date calculation.

Two-Factor Authentication & Activity Logs: Two-factor authentication would be added to strengthen login security, and user activity logs would be maintained to monitor account actions and detect suspicious behavior.

Improved Medicine Browsing: Pagination or infinite scroll would be implemented along with real-time inventory updates to improve performance and enhance the medicine browsing experience.

References

[1] PHP Manual. (2025). *Password hashing functions: password_hash() and password_verify()*. https://www.php.net/manual/en/ref.password.php

[2] PHP Manual. (2025). *Sessions in PHP*. https://www.php.net/manual/en/book.session.php

[3] MDN Web Docs. (2025). *Fetch API*. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[4] OWASP Foundation. (2024). *Unrestricted File Upload*. https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload